

# Autenticação

SEG786203 – CST em Análise e Desenvolvimento de Sistemas

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

# Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](https://creativecommons.org/licenses/by/4.0/)

# Introdução

# Objetivos da aula

- Compreender os conceitos sobre autenticação
- Identificar os principais métodos de autenticação de usuários
- Identificar os principais métodos de autenticação entre sistemas

# Política e mecanismo

## ■ Política

- *O que* é permitido ou negado
- Diretrizes, regras, procedimentos, padrões ou critérios

## ■ Mecanismo

- *Como* a política é implementada, aplicada e executada
- Método, ferramenta ou procedimento que implementa a política

# Identidade digital, autenticação e prova de identidade

- **Identidade digital** é a representação única de uma entidade usada para identificá-la em uma transação eletrônica
- **Autenticação** é o processo de verificar se a entidade realmente controla autenticadores associados a identidade digital

# Autenticadores são divididos em fatores

## Fatores de autenticação

Senha

\*\*\*\*\*

Aquilo que você sabe

Aquilo que você possui



Aquilo que você é

### ■ Conhecimento

- senha, PIN

### ■ Posse

- token, cartão, celular

### ■ Inerência

- características biométricas

# Autenticadores são divididos em fatores

## Fatores de autenticação

Senha

\*\*\*\*\*

Aquilo que você sabe

Aquilo que você possui



Aquilo que você é

### ■ Conhecimento

- senha, PIN

### ■ Posse

- token, cartão, celular

### ■ Inerência

- características biométricas

## Autenticação multifator (MFA)

Combina dois ou mais fatores de autenticação para tornar mais robusto o processo de autenticação



# Autenticação baseada em conhecimento I

## Senha

- Criada em 1961 por Fernando Corbató no MIT, para permitir que vários usuários acessassem o mesmo computador ao mesmo tempo
  - Projeto *Compatible Time-Sharing System* (CTSS)
- Ponto de partida para a criação dos atuais sistemas de autenticação baseados em senha

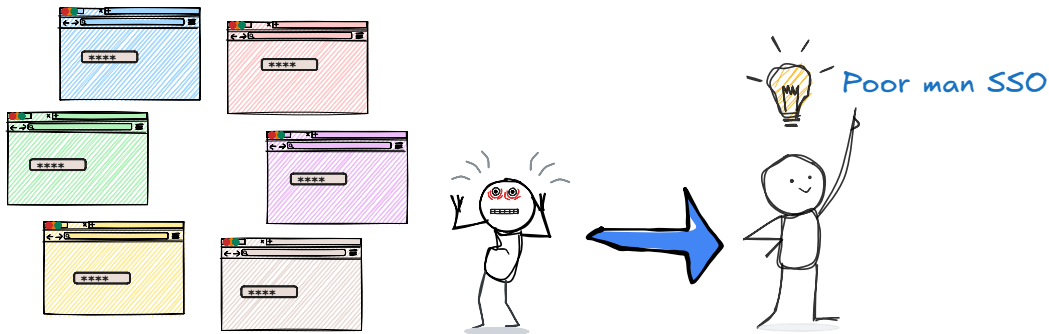


Fonte: <https://www.bbc.com/news/technology-48988091>

# Autenticação baseada em conhecimento II

## Senha

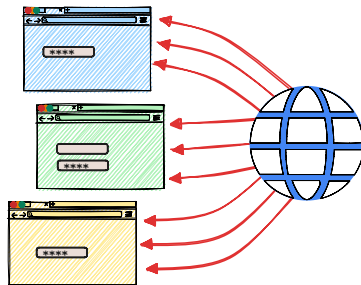
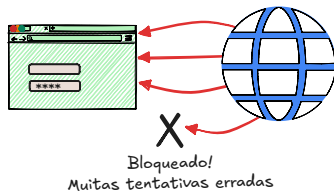
- Usuários tendem a usar **senhas fáceis** ou **reutilizar senhas** em vários serviços, o que aumenta o risco de comprometimento



# Autenticação baseada em conhecimento III

## Senha

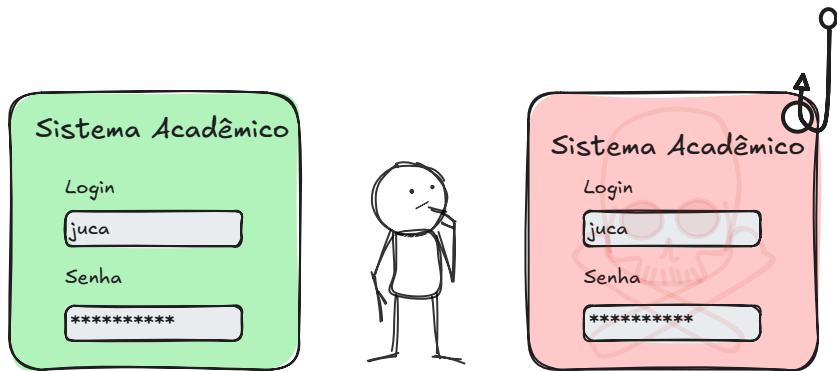
- Podem ser comprometidas por ataques de força bruta ou dicionário



# Autenticação baseada em conhecimento IV

## Senha

- Usuários podem ser vítimas de ataques de *phishing*



# Autenticação baseada em conhecimento V

## Senha

Senhas acabaram se tornando um pesadelo

Fernando Corbató, 2014.



Fonte: [https://en.wikipedia.org/wiki/Fernando\\_Corbat%C3%B3](https://en.wikipedia.org/wiki/Fernando_Corbat%C3%B3)

# Quais as características de uma boa senha?

- **Não usar sequências, repetições, dados pessoais ou palavras comuns?**
  - 123456, qwerty, 111111, Juca@2001, 10042001, password, incorrect
- **Combinar letras maiúsculas e minúsculas, números e símbolos?**
  - P@ssw0rd!, 123de0liveira4!
- **Criar um padrão para facilitar a memorização?**
  - Juca@2001-Gmail\*, Juca@2001-Facebook\*, Juca@2001-Ifsc!
- **Usar senhas longas?**
  - 12, 16, 20 caracteres?

# Entropia de uma senha I

## O que é entropia de uma senha?

Medida matemática que indica o grau de imprevisibilidade ou aleatoriedade de uma senha

- Reflete o número de possibilidades que precisam ser testadas até encontrar a senha correta
- Quanto maior a entropia, mais difícil é para um atacante adivinhar a senha por força bruta
- Senhas curtas, previsíveis ou baseadas em padrões comuns possuem baixa entropia

# Entropia de uma senha II

- A **entropia teórica é expressa em *bits*** e calculada com base no tamanho da senha e no número de caracteres possíveis que podem ser usados

$$E = \log_2(R^L) = L \times \log_2(R) \quad (1)$$

- Onde:

- $E$  é a entropia,
- $R$  é o número de caracteres possíveis,
- $L$  é o comprimento da senha e
- $\log_2$  para converter o número total de possibilidades em *bits*

## Nota

$\log_2(x)$  indica a potência à qual 2 deve ser elevado para resultar em  $x$ . Por exemplo,  $\log_2(8) = 3$  porque  $2^3 = 8$ . Pode ser calculado como  $\log_2(x) = \frac{\log_{10}(x)}{\log_{10}(2)}$



# Entropia de uma senha I

$$E = \log_2(R^L) = L \times \log_2(R)$$

- Uma senha de 4 dígitos numéricos tem  $\log_2(10^4) = 13,29$  *bits* de entropia
- Seriam necessárias cerca de  $2^{13,29} \approx 10.000$  tentativas para descobrir a senha por força bruta

| Tamanho da senha | Caracteres possíveis            | Combinações | Entropia ( <i>bits</i> ) |
|------------------|---------------------------------|-------------|--------------------------|
| 4                | 0..9 (10)                       | $10^4$      | 13,29                    |
| 4                | a..z (26)                       | $26^4$      | 18,80                    |
| 4                | a..z, A..Z, 0..9 (62)           | $62^4$      | 23,81                    |
| 4                | a..z, A..Z, 0..9, símbolos (94) | $94^4$      | 26,22                    |
| 11               | a..z, A..Z, 0..9, símbolos (94) | $94^{11}$   | 73,88                    |

# Entropia de uma senha II

$$E = \log_2(R^L) = L \times \log_2(R)$$

| Tamanho da senha | Número de caracteres possíveis  | Combinações possíveis | Entropia ( <i>bits</i> )  |
|------------------|---------------------------------|-----------------------|---------------------------|
| 4                | 0..9 (10)                       | $10^4$                | $\log_2(10^4) = 13,29$    |
| 4                | a..z (26)                       | $26^4$                | $\log_2(26^4) = 18,80$    |
| 4                | a..z, A..Z, 0..9 (62)           | $62^4$                | $\log_2(62^4) = 23,81$    |
| 4                | a..z, A..Z, 0..9, símbolos (94) | $94^4$                | $\log_2(94^4) = 26,22$    |
| 11               | a..z, A..Z, 0..9, símbolos (94) | $94^{11}$             | $\log_2(94^{11}) = 73,88$ |

## Calcule a entropia teórica das senhas

- 1 Qual a entropia teórica da senha Juca@2001?
- 2 Qual a entropia teórica de uma de suas senhas?

# Política de senhas

Segundo (Grassi; Garcia; Fenton, 2020) e sua revisão de 2024<sup>1</sup>

- **DEVE** ter no mínimo 8 caracteres, mas **DEVERIA** ter no mínimo 15 caracteres
- O tamanho máximo **DEVERIA** ser de pelo menos 64 caracteres
- **DEVERIA** permitir todos os caracteres ASCII, incluindo espaços, e **DEVERIA** aceitar todos os caracteres UNICODE
- **NÃO DEVE** impor restrições de composição arbitrárias (e.g, mistura de letras, números, símbolos)
- **NÃO DEVE** exigir a troca periódica de senhas, exceto quando houver evidência de comprometimento da senha

---

<sup>1</sup> <https://pages.nist.gov/800-63-4/sp800-63b.html#password>

# Força da senha

Indica o quão difícil é para ser descoberta

- **Zxcvbn**<sup>2</sup> é uma biblioteca que mede a força de uma senha e está disponível em várias linguagens de programação
- Faça um teste em <https://bitwarden.com/password-strength>



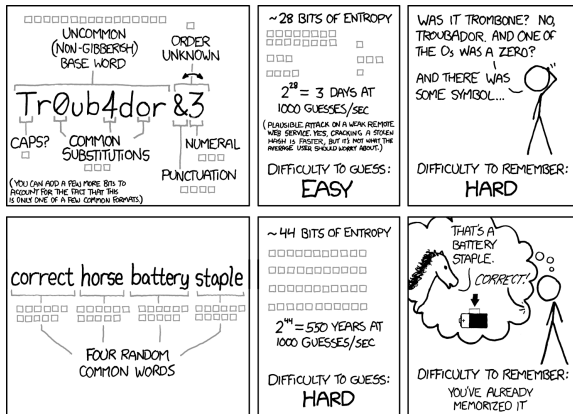
Fonte: <https://www.codehim.com/bootstrap/bootstrap-5-password-strength-meter>

---

<sup>2</sup><https://github.com/dropbox/zxcvbn>

# Política de senhas ruim resulta em senhas ruins

Explicação em [https://www.explainxkcd.com/wiki/index.php/936:\\_Password\\_Strength](https://www.explainxkcd.com/wiki/index.php/936:_Password_Strength)



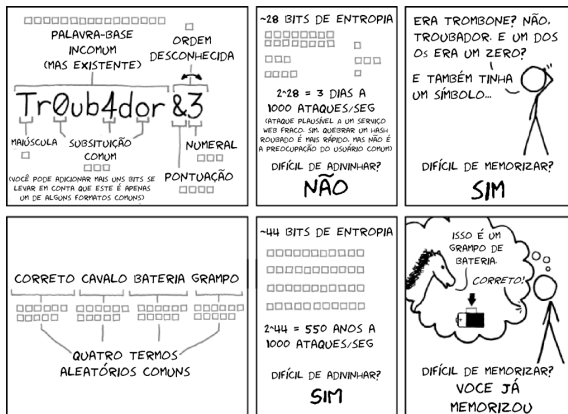
THROUGH 20 YEARS OF EFFORT, WE'VE SUCCESSFULLY TRAINED EVERYONE TO USE PASSWORDS THAT ARE HARD FOR HUMANS TO REMEMBER, BUT EASY FOR COMPUTERS TO GUESS.

- Cadeia aleatória de 11 caracteres teria 73,88 *bits*
- O padrão "palavra no dicionário + número ou símbolos", tem 28 *bits*
- O conjunto de palavras comuns tem 2048 símbolos, logo  $\log_2(2048^4) = 44$ 
  - Palavras que as pessoas escolhem para senhas são mais previsíveis

Fonte: <https://xkcd.com/936>

# Política de senhas ruim resulta em senhas ruins

Explicação em [https://www.explainxkcd.com/wiki/index.php/936:\\_Password\\_Strength](https://www.explainxkcd.com/wiki/index.php/936:_Password_Strength)



EM 20 ANOS, NOS EMPENHAMOS EM TREINAR AS PESSOAS A USAR SENHAS QUE SÃO DIFÍCEIS DE MEMORIZAR, MAS FÁCEIS DE COMPUTADORES DESCOBRIREM

Fonte: <https://cacovsky.wordpress.com/2011/12/29/>

- Cadeia aleatória de 11 caracteres teria 73,88 bits
- O padrão “palavra no dicionário + número ou símbolos”, tem 28 bits
- O conjunto de palavras comuns tem 2048 símbolos, logo  $\log_2(2048^4) = 44$ 
  - Palavras que as pessoas escolhem para senhas são mais previsíveis

# Armazenamento de senhas

# Como armazenar a senha dos usuários?

## Implementação de autenticação baseada em senha

- Texto claro
- Cifradas com algoritmos simétricos
- Resumo criptográfico (*hash*)
- Resumo criptográfico com *salt*



# Armazenamento de senha

## Texto claro

- A senha é armazenada em texto claro no banco de dados, arquivo ou outro meio
- Qualquer pessoa com acesso ao banco de dados pode ver a senha, incluindo administradores de banco de dados

```
juca:Juca@2001  
maria:Senha123  
pedro:123456
```

# Armazenamento de senha

## Cifrada com algoritmo simétrico

- A senha é cifrada usando um algoritmo simétrico (e.g. AES) antes de ser armazenada
  - A chave secreta deve ser mantida em segredo e não deve ser armazenada junto com a senha
- Este método permite que a senha seja recuperada, ou seja, com a chave secreta **é possível recuperar a senha original**

```
juca:a465f79b5d8ec47fa32110dfb48c769e  
maria:1a2b3c4d5e6f7a8b9cadbecfd0e1f203
```

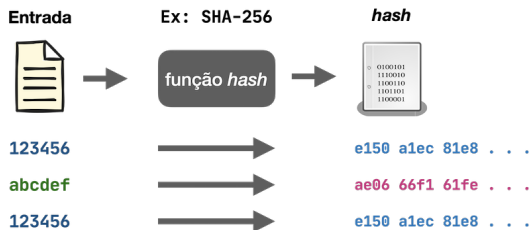
### ⚠ Cuidado

Atualmente não é recomendado armazenar senhas cifradas com algoritmos simétricos

# Armazenamento de senha

## Resumo criptográfico (*hash*)

- Funções de *hash* são de **sentido único**
  - Não é possível recuperar a informação original a partir do *hash*
- Um ponto ruim é que uma mesma entrada sempre gera o mesmo *hash*
  - Se dois usuários escolherem a mesma senha, então o *hash* será o mesmo



# Armazenamento de senha

## Resumo criptográfico com *salt*

- O ***salt*** é uma sequência aleatória única que é concatenada à senha antes de calcular o *hash*

$$\text{hash} = \text{função\_hash}(\text{senha} \parallel \text{salt}) \quad (2)$$

- O *salt* deve ser armazenado junto com o *hash* para que a senha possa ser verificada

**username:\$alg\$salt\$hash**

**juca:\$5\$sPKa/BseZ666/2ME\$LQX0pijAqyPEYonvo.pp9bAhYcMdXUWS7pBqJX0ZAI6**  
**mari:\$5\$TZMZS7Z4hfG8w9/4\$QXu9fhomzMvEkWNieN1ZoY.1eabhDLUgcCJssImf1e5**

Figura: Exemplo do arquivo `/etc/shadow` no Linux. Ambos usuários escolheram a senha 123456 mas possuem *salts* diferentes

# Armazenamento de senha

## Quais algoritmos usar?

- Funções de dispersão criptográfica são **projetadas para serem rápidas**, o que as torna vulneráveis a ataques de força bruta
  - e.g. MD5, SHA-1, SHA-256, SHA-512
- Para senhas, o ideal é usar funções de **hash lentas** que consomem muitos recursos (CPU ou memória)
  - e.g. bcrypt, scrypt, PBKDF2, Argon2
- Debian 11 e derivados usam o algoritmo **yescrypt**
  - Antes já foi usado *crypt* com DES, MD5 e por último SHA-512
- **Argon2** foi o escolhido pela *Password Hashing Competition*<sup>3</sup>
  - **Lyra2** (da USP) e **yescrypt** ficaram entre os finalistas

---

<sup>3</sup><https://password-hashing.net>

# Senhas do usuários no Linux

Arquivo `/etc/passwd` contém informações sobre os usuários

```
aluno:x:1000:1000:Aluno do ADS:/home/aluno:/bin/bash
```

- `aluno` é o nome do usuário
- `x` indica que a senha está armazenada em `/etc/shadow`
- `1000` é o *user ID* e *group ID*
- `Aluno do ADS` é o nome completo do usuário
- `/home/aluno` é o diretório do usuário
- `/bin/bash` é a *shell* padrão

# Senhas do usuários no Linux

Arquivos `/etc/shadow` contém as senhas dos usuários

```
aluno:$y$j9T$.0gygyle3hBKaEqRXD9G90$klVKdyeRxjdsENuTo9wMQ4kN6UCzfx4wBp5C2RDGar3  
:19880:0:99999:7:::
```

- `aluno` é o nome do usuário
- `$y$` indica que o algoritmo usado é `yescrypt`
- `$y$j9T$.0gygyle3hBKaEqRXD9G90` é o *salt*
- `$klVKdyeRxjdsENuTo9wMQ4kN6UCzfx4wBp5C2RDGar3` é o *hash* da senha
- `19880` data da última mudança de senha em dias desde 01/01/1970
- `0` é o número de dias que o usuário deve esperar para mudar a senha
- `99999` é o número de dias que a senha é válida
- `7` é o número de dias antes do usuário ser avisado que a senha vai expirar

# Gerar *hash* de senha no Linux com `mkpasswd`

Comando `mkpasswd` faz parte do pacote `whois`

## ■ Gerando *hash* de senha com diferentes algoritmos

```
# Algoritmos disponíveis: DES, MD5, SHA-256, SHA-512, yescrypt (mkpasswd -m help)
mkpasswd -m DES 123456
# saída: B9.75nVORnpzQ
mkpasswd -m MD5 123456
# saída: $1$9L8oTH9e$xMYUa5Ic3oHb/MMG4n06i.
mkpasswd -m SHA-256 123456
# saída: $5$dqZjNvmwsZdkVmZr$sGzvs/zDeELQpwbvNpMnTMR2exKePljxe28d0j5cq.
mkpasswd -m yescrypt 123456
# saída: $y$j9T$udU2MHIBBBqw09CSVUfTQ0$dVhi/fxp.hPl4a.uw0D8.FDdvRyF8pyI3n/CYxRpgh2
```

## ■ Gerando *hash* de senha com *salt* definido

```
mkpasswd -m SHA-256 -S 'dqZjNvmwsZdkVmZr$' 123456

mkpasswd -m yescrypt -S '$y$j9T$udU2MHIBBBqw09CSVUfTQ0' 123456
```



# Aumentar a segurança do armazenamento de senhas

## Usar *salt* e *pepper*

- O ***salt*** evita que senhas iguais tenham o mesmo hash, mas não impede ataques de força bruta contra uma base de senhas comprometidas
  - Atacante terá acesso ao *salt* e poderá gerar o hash de uma senha candidata
- O ***pepper*** é um valor aleatório e secreto que é concatenado à senha antes de calcular o hash
  - É armazenado separadamente do *hash* e do *salt* (e.g. cofre de senhas)
  - Se a base de dados de senhas for comprometida, o pepper não estará disponível para o atacante

$\text{hmac-sha256}(\text{pepper} \parallel (\text{sha256}(\text{senha} \parallel \text{salt})))$  (3)

# Ataques a senhas

# Principais ataques a senhas

- Roubo da base de senhas
- Ataque de força bruta e ataque de dicionário
- Ataque de tabela arco-íris (*rainbow tables*)
- *Keyloggers* e *screenloggers*
- Captura de tráfego (*password sniffing*)
- Engenharia social
  - *Phishing*, *vishing* (voz), *smishing* (SMS), MITM, redefinição de senha

# Roubo da base de senhas

- O atacante busca obter acesso a uma base de dados que armazena senhas para descobrir senhas de usuários
  - Se a senha estiver armazenada em texto claro, o atacante pode ver as senhas diretamente
  - Se a senha estiver armazenada como um *hash*, o atacante pode tentar quebrar o *hash* usando ataques de força bruta ou dicionário
  - Se a senha estiver armazenada com *salt* e *pepper*, o atacante ainda pode tentar quebrar o *hash*, mas terá mais dificuldade
- O serviço “*Have I Been Pwned?*”<sup>4</sup> permite verificar se uma senha foi comprometida em vazamentos de dados

---

<sup>4</sup><https://haveibeenpwned.com/Passwords>

# Exemplos de vazamento de bases de senhas ou dados

- **eBay** (2014)
- **Ashley Madison** (2015)
- **LinkedIn** (2012, 2016, 2021,...)
- **Collection #1** (2019) – 773 milhões de emails e 21 milhões de senhas únicas
- **Facebook** (2021) – 530 milhões de contas
- **Troy Hunt's Mailchimp List accounts**<sup>5</sup> (2025) – 16mil emails
- **XP Investimentos** (2025) – dados cadastrais, saldo e posição de investimentos

---

<sup>5</sup>Troy Hunt é o criador do *Have I Been Pwned?*.

<https://www.troyhunt.com/a-sneaky-phish-just-grabbed-my-mailchimp-mailing-list/>

# Ataque de força bruta

## O atacante tenta todas as combinações possíveis de senhas

- Uma senha de 9 caracteres alfanuméricos tem  $\approx 1,35 \times 10^{16}$  combinações
  - Se o atacante puder testar 100.000 senhas por segundo, levaria cerca de 4 anos para descobrir a senha
- O uso de heurísticas pode reduzir o espaço de busca, considerando o comportamento humano ao criar senhas
  - Exemplo: *Juca2001* de 4 anos para 40 minutos
- *John the Ripper* e *Hashcat* são ferramentas populares para quebra de senhas
  - Hashcat + NVIDIA GeForce RTX 4090<sup>6</sup>, é possível testar  $7,4 \times 10^9$  senhas por segundo

---

<sup>6</sup><https://gist.github.com/Chick3nman/32e662a5bb63bc4f51b847bb42222fd>

# Ataque de dicionário

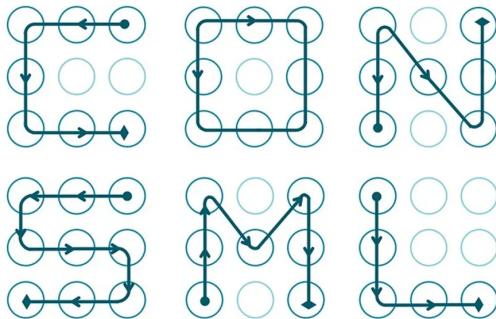
## O atacante tenta senhas comuns ou palavras do dicionário

- O atacante pode usar listas de senhas comprometidas ou dicionários personalizados para aumentar a eficácia do ataque
  - O uso de dicionários pode reduzir significativamente o tempo necessário para quebrar uma senha
- Dicionários são listas de palavras, frases ou combinações de caracteres que são usadas para tentar adivinhar a senha
  - Nome de pessoas, animais, cidades, times de futebol, filmes, músicas
  - Datas de nascimento, casamento, placas de carro, telefones
  - 123456, password, password1, iloveyou, fera@123

# Ataque de dicionário

## Padrão para desbloquear celular Android

- Marte Løge<sup>7</sup> em 2015 apresentou um estudo com mais de 4mil padrões de desbloqueio e concluiu que são **extremamente previsíveis**



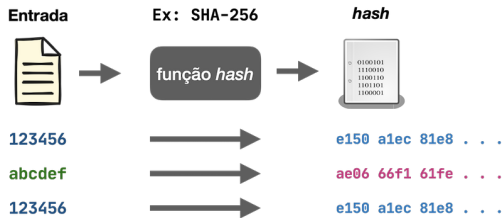
<sup>7</sup><https://arstechnica.com/information-technology/2015/08/new-data-uncovers-the-surprising-predictability-of-android-lock-patterns/>



# Ataque de tabela arco-íris (*rainbow tables*)

O projeto *Free Rainbow Tables*<sup>8</sup> disponibiliza tabelas arco-íris para *download*

## ■ Tabelas de *hash* pré-computadas para gerar senhas candidatas



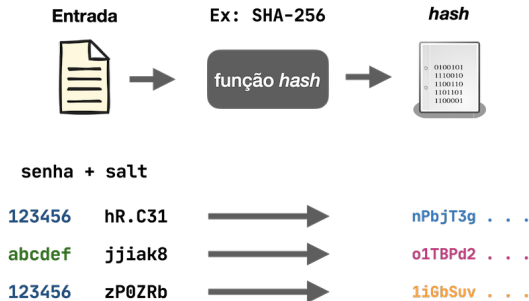
- São muito mais rápidas de usar do que calcular o *hash* de cada senha candidata
- Tabelas de *hash* são grandes, mas contam com algoritmos de redução que permitem reduzir seu tamanho

<sup>8</sup><https://project-rainbowcrack.com>

# Ataque de tabela arco-íris (*rainbow tables*)

## Como se proteger?

- Uso de *salt*, combinado ou não com *pepper*, além de algoritmos lentos, é suficiente para proteger contra esse tipo de ataque



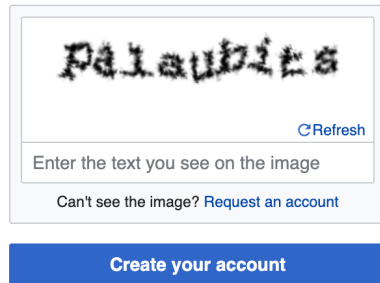
# Como aumentar a segurança do processo de autenticação

# Autenticação baseada em senha

## Como aumentar a segurança?

- **Bloqueio de contas** após várias tentativas de autenticação falhas
- **Tempo de espera** entre tentativas de autenticação
- **Captcha**<sup>a</sup> para evitar ataques automatizados
- **Autenticação de dois fatores** para aumentar a robustez
- **Política de senhas** para aumentar a entropia

CAPTCHA Security check ([what is this?](#))



Fonte: Mover of molehills, Wikimedia

---

<sup>a</sup><https://blog.cloudflare.com/end-cloudflare-captcha>

# Autenticação multifator ou de dois fatores

- **Autenticação multifator** exige a combinação de dois ou mais fatores de categorias distintas para autenticar o usuário
  - Exemplo: senha + código enviado por SMS ou aplicativo autenticador
- Aumenta significativamente a segurança do processo de autenticação
  - Um fator baseado em conhecimento **pode ser comprometido remotamente**
  - Um fator baseado em posse normalmente só pode ser comprometido fisicamente

# Implementação de autenticação baseada em senha

*"Senhas acabaram se tornando um pesadelo", Fernando Corbató (2014)*

## ■ Vantagens

- Usuário não precisa de equipamento adicional
- Fácil de ser alterada
- Fácil de ser memorizada (quando não complexa)

## ■ Desvantagens

- Vulnerável a ataques de força bruta, *phishing*, *shoulder surfing*
- Difícil de ser memorizada
- Difícil de ser digitada em dispositivos móveis
- Usuário pode reutilizar a senha em diferentes serviços
- Senhas podem ser comprometidas em vazamentos de dados

# Gerenciador de senhas

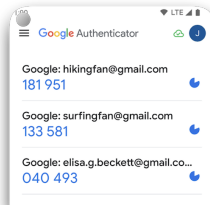
- Software que permite armazenar senhas de forma segura
  - Gerador de senhas com entropia alta e únicas para cada serviço
  - Preenche automaticamente formulários de *login* em *sites*
  - Sincronização de senhas entre dispositivos
- Exemplos
  - *Bitwarden, 1Password, KeePass, ProtonPass*
  - Google Password Manager
  - Apple Password

## ⚠ Atenção

Você deve usar um gerenciador de senhas confiável e seguro. Usar uma senha mestre forte é essencial

# Autenticação baseada em posse

- **Certificado digital** armazenado em cartão inteligente ou *token*
  - e-CPF, e-CNPJ
- **Senha de uso único**
  - token físico ou aplicativo
  - SMS ou ligações telefônicas
- **Chave de segurança hardware** conectado ao computador
  - FIDO2 YubiKey





# Autenticação de usuário com certificado digital

Fator de posse, algo que o usuário tem

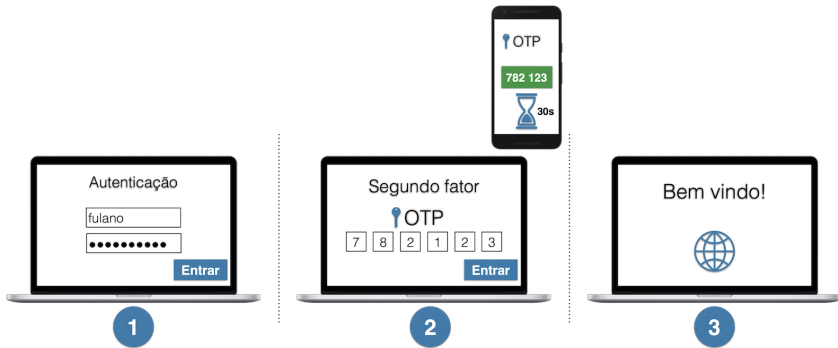
- **Certificado ICP-Brasil A1** é armazenado em arquivo digital
  - Deve ser carregado no navegador ou no sistema cliente para autenticação
- **Certificado ICP-Brasil A3** é armazenado em cartão inteligente ou *token*
  - É necessário driver para o dispositivo



Fonte: <https://certificadodigitalspc.org.br>

# Senha de uso único

*One-Time Password (OTP)*



# HMAC-based One-Time Password, HOTP<sup>9</sup>

Senha de uso único

**Segredo  
compartilhado**



**Fator de  
incremento**



- HMAC é aplicado ao segredo compartilhado (K) e ao contador (C)
  - O segredo compartilhado é uma chave secreta de 128 bits
  - O contador é um número inteiro que é incrementado a cada autenticação por ambas as partes
- O contador deve ser sincronizado em caso de descompasso

$$HOTP(K, C) = \text{truncate}(\text{HMAC-SHA-1}(K, C)) \quad (4)$$

<sup>9</sup><https://datatracker.ietf.org/doc/html/rfc4226>

# Time-based One-Time Password, TOTP<sup>10</sup>

Senha de uso único

**Segredo  
compartilhado**



**Fator de  
incremento**



- TOTP usa o instante atual ( $T$ ) como contador
  - Número de segundos desde 01/01/1970 (era Unix) dividido pelo intervalo de tempo (geralmente 30 segundos)
- As senhas geradas são válidas por um curto período de tempo (30s)
- Requer que o relógio de ambas as partes estejam sincronizados

$$TOTP(K) = HOTP(K, \text{floor}(T/\text{intervalo})) \quad (5)$$

---

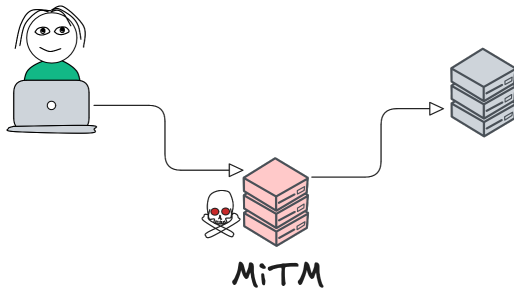
<sup>10</sup><https://datatracker.ietf.org/doc/html/rfc6238>

# Senhas de uso único

## Discussão

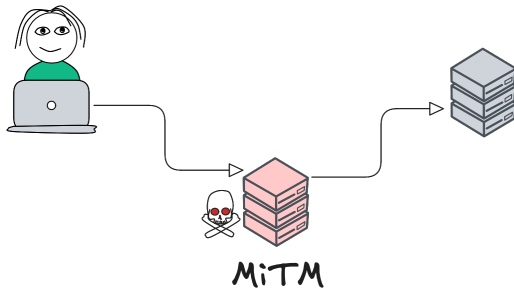
- **TOTP é preferível a HOTP por ser mais seguro**
  - Tempo de vida da senha (30s) limita a janela de oportunidade para um atacante
- **O segredo (*seed*) deve ser armazenado de forma segura**
  - Alguém que tenha acesso ao segredo pode gerar senhas válidas
  - Se o segredo for perdido por um dos lados, não será mais possível autenticar
- *Google Authenticator*, *Apple Password* e *Authy* permitem fazer *backup* dos segredos na nuvem
- Amazon KMS e Azure Key Vault podem ser usados para armazenar segredos no lado do servidor

# Ataque do homem do meio (*Man-in-the-Middle*, MitM)



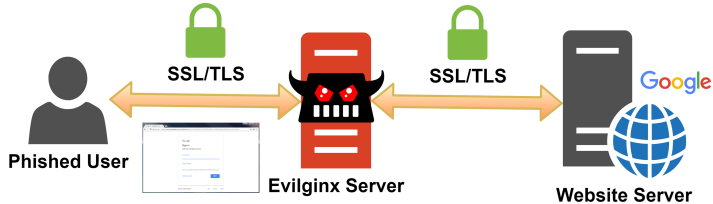
- O atacante fica entre o usuário e o serviço, podendo interceptar e modificar a comunicação entre eles
  - Pode ser feito por meio de redes Wi-Fi públicas, *phishing* ou *malware*

# Ataque do homem do meio (*Man-in-the-Middle*, MitM)



- O atacante fica entre o usuário e o serviço, podendo interceptar e modificar a comunicação entre eles
  - Pode ser feito por meio de redes Wi-Fi públicas, *phishing* ou *malware*
- O TLS e o OTP me deixam imune a ataques MitM! **Será?**

# Ataque do homem do meio contra OTP



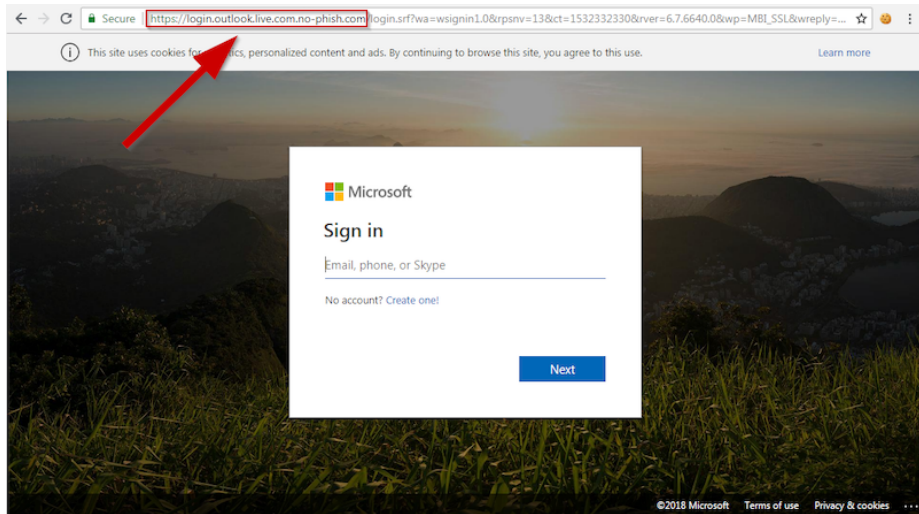
Fonte: <https://breakdev.org>

- **Evilginx**<sup>11</sup> é uma ferramenta MitM que captura credenciais e cookies de sessão, **permitindo contornar a autenticação com dois fatores (OTP)**

<sup>11</sup><https://github.com/kgretzky/evilginx2>

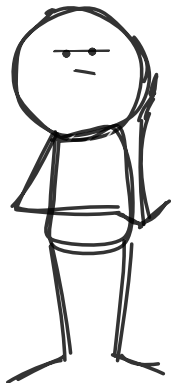


# Ataque do homem do meio contra OTP



Fonte: <https://breakdev.org>

# Ataque do homem do meio contra OTP



■ *Eu jamais cairia em um ataque desses!*

# Ataque do homem do meio contra OTP



- *Eu jamais cairia em um ataque desses!*
- *Eu consigo identificar um site falso de longe!*

# Ataque do homem do meio contra OTP



- *Eu jamais cairia em um ataque desses!*
- *Eu consigo identificar um site falso de longe!*
- *Eu não clicaria em um link desses, mesmo se estivesse distraído ou acessando pelo celular!*

# Ataque do homem do meio contra OTP

- Troy Hunt mantenedor do site *Have I Been Pwned?*
  - O site permite que usuários verifiquem se seus emails ou senhas foram comprometidas em vazamentos de dados
- Em 2025<sup>11</sup>, ele foi vítima de um ataque *phishing* e MitM

*"Isso aconteceu logo depois de voltar de uma palestra no National Cyber Security Centre (UK) onde falei como podemos promover melhor as chaves de acesso (passkeys), em parte devido à sua natureza resistente a phishing."*

---

<sup>11</sup> <https://www.troyhunt.com/a-sneaky-phish-just-grabbed-my-mailchimp-mailing-list/>

# Ataque do homem do meio contra OTP



- Talvez eu deva afirmar:
  - **Eu AINDA não caí em um ataque desses!**

# Chaves de acesso (FIDO *Passkeys*)

Foco na segurança, privacidade e usabilidade

Autenticação remota de usuários sem senha e **sem atrito**



Baseado em **criptografia de**  
**chave pública**

Um **par de chaves** por  
provedor de **serviço**



Seguro contra **vazamento**  
**de dados**

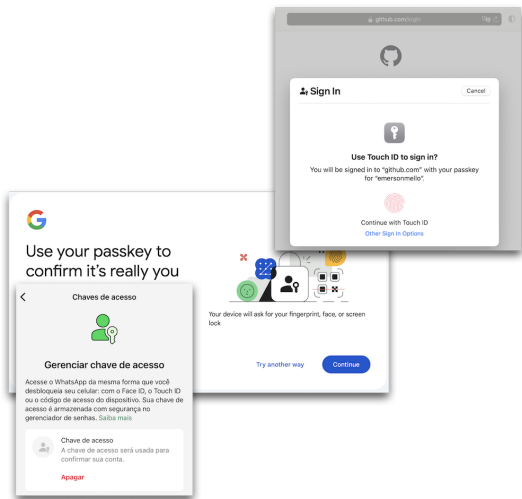
Seguro contra **phishing**  
Garante a **privacidade**



*Passkeys* **salvas nos**  
**dispositivos** do usuário

**Autenticação biométrica**  
**local** no dispositivo

# Chaves de acesso (FIDO Passkeys) I



- Usadas como **segundo fator** ou **único fator** (*passwordless*)
- Permitem autenticação *usernameless*
- Permitem autenticação cruzada (ex: usar *passkey* do celular no PC)



# Chaves de acesso (FIDO *Passkeys*) II

- Autenticadores FIDO podem ser dispositivos (celular, computador) ou tokens físicos
- Chaves de acesso são sincronizadas entre dispositivos do mesmo ecossistema
- A sincronização entre diferentes ecossistemas está prevista para até o final de 2025<sup>a</sup>

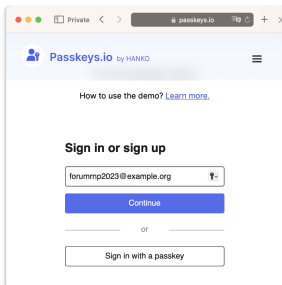
<sup>a</sup><https://fidoalliance.org/specs/cx/cxf-v1.0-rd-20250313.html>



# Chaves de acesso (FIDO *Passkeys*)

Processo de registro: você pode fazer esse demo em <https://www.passkeys.io/>

Escolha o nome de usuário



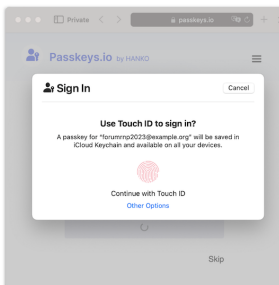
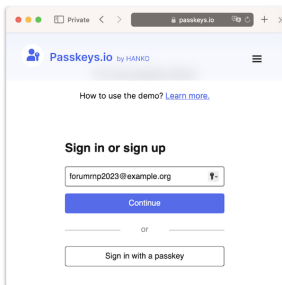
The screenshot shows a web browser window with the address bar displaying "passkeys.io". The page header includes the "Passkeys.io by HANKO" logo and a menu icon. Below the header, there is a link "How to use the demo? [Learn more.](#)". The main heading is "Sign in or sign up". Under this heading, there is a text input field containing the email "forummp2023@example.org" and a small icon of a person. Below the input field is a blue button labeled "Continue". Below the "Continue" button, the word "or" is centered between two horizontal lines. At the bottom, there is a button labeled "Sign in with a passkey".

# Chaves de acesso (FIDO Passkeys)

Processo de registro: você pode fazer esse demo em <https://www.passkeys.io/>

Escolha o nome de usuário

Autentique com sua biometria



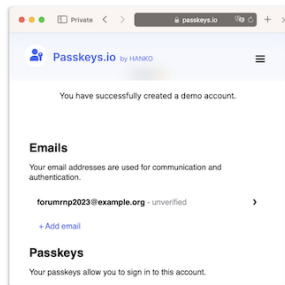
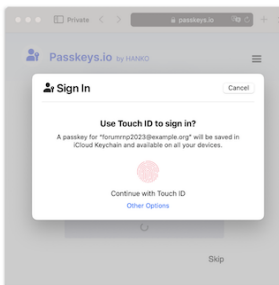
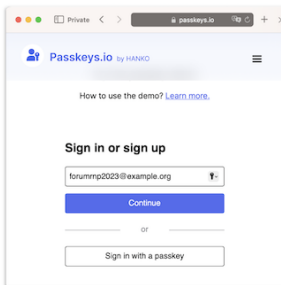
# Chaves de acesso (FIDO Passkeys)

Processo de registro: você pode fazer esse demo em <https://www.passkeys.io/>

Escolha o nome de usuário

Autentique com sua biometria

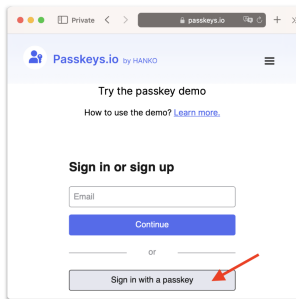
Bem vindo!



# Chaves de acesso (FIDO Passkeys)

## Processo de autenticação *usernameless* e *passwordless*

Clique para autenticar

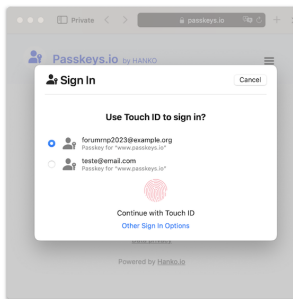
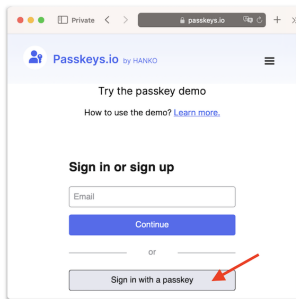


# Chaves de acesso (FIDO Passkeys)

Processo de autenticação *usernameless* e *passwordless*

Clique para autenticar

Escolha a passkey desejada



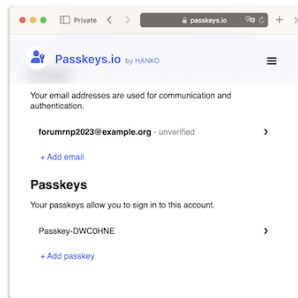
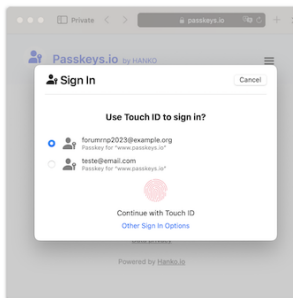
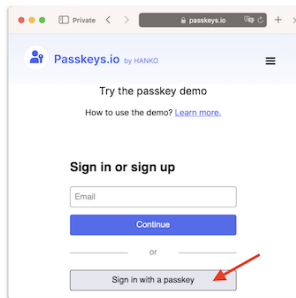
# Chaves de acesso (FIDO Passkeys)

Processo de autenticação *usernameless* e *passwordless*

Clique para autenticar

Escolha a passkey desejada

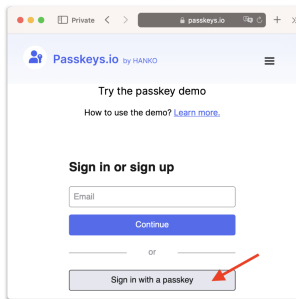
Bem vindo!



# Chaves de acesso (FIDO Passkeys)

## Processo de autenticação cruzada

Clique para autenticar



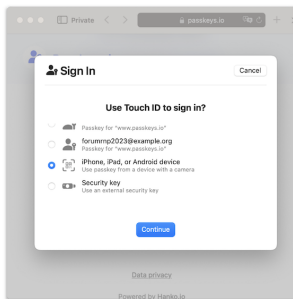
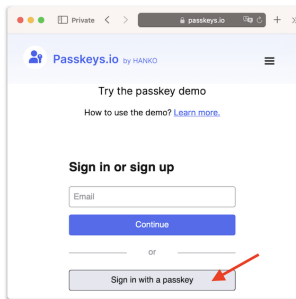


# Chaves de acesso (FIDO Passkeys)

## Processo de autenticação cruzada

Clique para autenticar

Escolha a passkey desejada



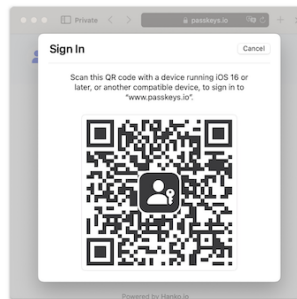
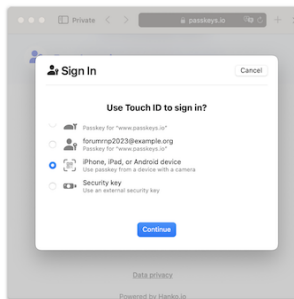
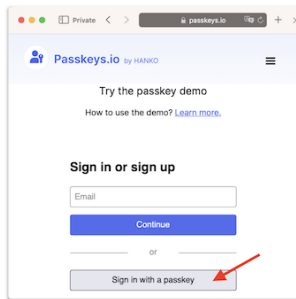
# Chaves de acesso (FIDO Passkeys)

## Processo de autenticação cruzada

Clique para autenticar

Escolha a passkey desejada

Use a câmera do celular



# Chaves de acesso (FIDO *Passkeys*)

Suporte a autenticadores em tokens físicos conectados via USB, NFC ou Bluetooth

|                   | Android 7+ | iOS 14.5+ | Windows 10 | macOS Catalina | macOS Big Sur | Desktop Linux |
|-------------------|------------|-----------|------------|----------------|---------------|---------------|
| Chrome            | Yes        | Yes       | Yes        | Yes            | Yes           | Yes           |
| Safari            | N/A        | Yes       | N/A        | Yes            | Yes           | N/A           |
| Firefox           | No         | Yes       | Yes        | Yes            | Yes           | Yes           |
| Brave             | No         | Yes       | Yes        | Yes            | Yes           | Yes           |
| Edge              | No         | No        | Yes        | Yes            | Yes           | Yes           |
| Internet Explorer | N/A        | N/A       | No         | N/A            | N/A           | N/A           |

Fonte: <https://webauthn.me/browser-support>

- Veja mais detalhes sobre suporte a chaves de acesso em navegadores e sistemas operacionais em <https://passkeys.dev/device-support>

# Chaves de acesso (FIDO Passkeys)

Suporte a autenticadores embarcados em dispositivos móveis e computadores

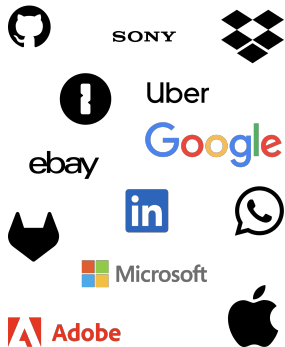
|                   | Android 7+ | iOS 14.5+ | Windows 10<br>(with Windows Hello) | macOS Catalina | macOS Big Sur | Desktop Linux |
|-------------------|------------|-----------|------------------------------------|----------------|---------------|---------------|
| Chrome            | Yes        | Yes       | Yes                                | Yes            | Yes           | -             |
| Safari            | N/A        | Yes       | N/A                                | No             | Yes           | N/A           |
| Firefox           | No         | Yes       | Yes                                | No             | No            | -             |
| Brave             | No         | Yes       | Yes                                | Yes            | Yes           | -             |
| Edge              | No         | Yes       | Yes                                | Yes            | Yes           | -             |
| Internet Explorer | N/A        | N/A       | No                                 | N/A            | N/A           | N/A           |

Fonte: <https://webauthn.me/browser-support>

- Veja mais detalhes sobre suporte a chaves de acesso em navegadores e sistemas operacionais em <https://passkeys.dev/device-support>

# Provedores de serviços com suporte às chaves de acesso

<https://passkeys.directory>

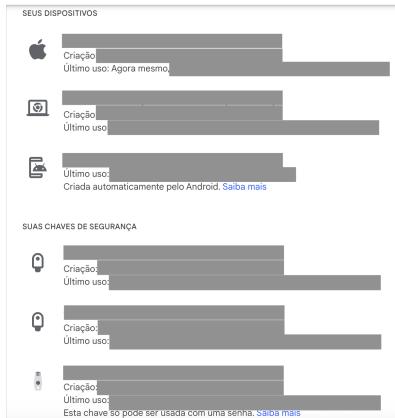


- Grandes provedores de serviços já estão adotando
- Usuários se beneficiam de uma experiência de autenticação mais segura e conveniente
- Instituições reduzem o risco de vazamentos de senhas e ataques de *phishing* e MitM

# Faça como eu!

Use chaves de acesso quando possível!

## ← Chaves de acesso e de segurança



- Tenho várias chaves de acesso associadas a minha conta Google
- E você?
  - <http://g.co/passkeys>

# Ok, eu quero ofertar chaves de acesso para meus usuários!

- Invoque a API *WebAuthn* dos navegadores<sup>11</sup>

```
const credential = await navigator.credentials.create({  
  publicKey: publicKeyCredentialCreationOptions  
});
```

- É desejado permitir o registro de mais uma chaves de acesso por usuário
  - one-to-many

---

<sup>11</sup><https://webauthn.io> tem bibliotecas para várias linguagens

# Autenticação baseada em posse

## Discussão

- Fator de posse é mais seguro que fator de conhecimento, pois dificulta ataques remotos
- Certificado digital é mais seguro que (senha + OTP)
  - São mais difíceis de serem usados, especialmente em dispositivos móveis
- OTP está suscetível a ataques de *phishing*, *malwares* no dispositivo do usuário e MITM
- Chaves de acesso (FIDO *Passkeys*) são mais seguras e mais fáceis de usar
  - Imunes a ataques de MITM



# Autenticação de usuários

Comentários de Schneier (2020)

- A autenticação consiste em uma troca entre segurança e usabilidade
  - Usuários, irritados por mecanismos de segurança, tendem a contorná-los
- Senhas oferecem péssima segurança, sendo adequadas apenas para aplicações não críticas
- A autenticação de usuários está cada vez mais desafiadora, ao passo que roubar credenciais de acesso está tornando-se mais fácil
- Rob Joyce (NSA), afirmou vulnerabilidades de dia zero são superestimadas e que é pelo roubo de credenciais que ele entra nas redes

# Autenticação entre serviços

# Autenticação entre serviços

Algumas definições fundamentais

# Formato da URI

`scheme://host:port/path?queryString#fragment`

- **scheme** – HTTP, HTTPS, FTP
- **host** – nome ou IP
- **port** – implícito ao esquema (p.e. 80 para http) ou explícito
- **path** – segmentos de texto delimitados por /
- **queryString** – lista de parâmetros (nome=valor) delimitados por &
- **fragment** – ponto particular dentro um documento

# JavaScript Object Notation (JSON)

Formato compacto, de padrão aberto independente, de troca de dados – <http://json.org>

- Um **objeto** é um conjunto não ordenado de pares chave:valor, encapsulados por chaves { e } e separados por vírgula

```
{  
  "nome": "Alice",  
  "sobrenome": "dos Santos",  
  "idade": 20,  
  "telefones": [  
    {  
      "tipo": "residencial",  
      "numero": "48 3381 2800"  
    },  
    {  
      "tipo": "celular",  
      "numero": "48 93381 2800"  
    }  
  ]  
}
```

- Strings são delimitadas por aspas
- Números são representados obrigatoriamente na forma decimal e seguem a sintaxe de representação do C, C++, Java
- Vetor é uma coleção ordenada de valores, delimitados por colchetes
  - Pode conter string, número, true, false, null, objeto ou vetor

# Autenticação entre serviços

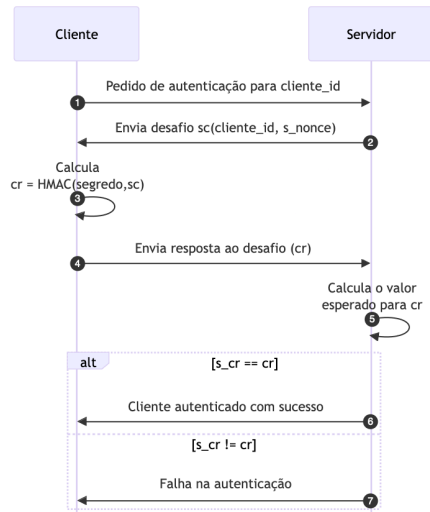
Autenticação em aplicações Web

# Autenticação entre serviços

- Os Serviços Web (*Web Services*) são uma forma de integração entre sistemas, permitindo exportar funcionalidades de um sistema para outro
  - APIs RESTful, SOAP, gRPC, GraphQL
- A autenticação entre serviços é necessária para garantir que o serviço solicitante é quem diz ser
- Há casos que é desejado a **autenticação mútua**, onde ambas as partes (cliente e servidor) devem se autenticar
  - Comportamento padrão do protocolo de autenticação usado no SSH e opcionalmente no TLS
  - Também aplicado no cenário da Internet das Coisas (IoT, *Internet of Things*)

# Protocolo de autenticação desafio e resposta

- Cliente e servidor compartilham previamente um segredo
- Servidor gera desafio
  - mensagem aleatória e única (*nonce*)
  - $\text{HMAC}(\text{segredo}, \text{cliente\_id} || s\_nonce)$
- Cliente responde com o desafio
  - $\text{HMAC}(\text{segredo}, sc)$





# Consumo de APIs que requerem autenticação

## Autenticação básica (*Basic Auth*) HTTP

- **Não é seguro** enviar credenciais em texto claro, então deve ser usado em conjunto com HTTPS
- Servidor avisa o cliente que a autenticação é necessária

```
HTTP/1.1 401 Unauthorized  
Date: Wed, 28 Apr 2025 07:28:00 GMT  
WWW-Authenticate: Basic realm="example.org"
```

- No pedido subsequente, cliente envia as credenciais codificadas em *base64* no cabeçalho *Authorization*

```
#Authorization: Basic base64(username:password)  
Authorization: Basic YWx1bm86bWluaGEgc2VuaGEgc2VjcmV0YQ==
```

# Consumo de APIs que requerem autenticação I

## Autenticação por *token* ou *API Key*

- Cadeia de caracteres gerada aleatoriamente, que pode ser usada para autenticar o cliente
- *API Keys* são geralmente opacas, ou seja, o servidor que gerou o *token* é o único que pode decodificá-lo
  - *GitHub Personal Access Token (PAT)* é um exemplo de *API Key*
- Devem ser armazenadas e transferidas de forma segura
  - Nunca deixe no código fonte e só envie sobre HTTPS
  - Faça uso de soluções como *Azure Key Vault*, *AWS Secrets Manager* ou *Google Secret Manager*

# Consumo de APIs que requerem autenticação II

## Autenticação por *token* ou *API Key*

- O cliente deve enviar o *token* em cada requisição, podendo ser enviado de diferentes formas

- No cabeçalho HTTP

```
GET /notas HTTP/1.1  
X-API-Key: 123456
```

- Como *query string* da URL

```
POST /notas?api_key=123456 HTTP/1.1
```

- Como um *cookie*

```
GET /notas HTTP/1.1  
Cookie: X-API-Key=123456
```

# Consumo de APIs que requerem autenticação III

## Autenticação por *token* ou *API Key*

- O campo usado no cabeçalho HTTP para enviar o *token* é definido pelo desenvolvedor da API
  - O mais comum em APIs Restful é usar o cabeçalho `Authorization` com o prefixo `Bearer`
  - Mas também pode ser usado o cabeçalho `X-API-Key` ou `X-Access-Token`

Código: Exemplo de requisição com *token* no cabeçalho

```
curl -L -H "Authorization: Bearer <TOKEN>" \  
https://api.github.com/repos/OWNER/REPO/commits
```

# JSON Web Token (JWT)<sup>12</sup>

- JWT é um padrão aberto que define um formato compacto para troca segura de informações entre partes como um objeto JSON
- Amplamente usado para autenticação e autorização em aplicações *web*, especialmente em APIs RESTful
- JWT é assinado digitalmente para garantir a integridade e autenticidade dos dados, podendo ser criptografado para garantir a confidencialidade
- Parte do *framework Javascript Object Signing and Encryption* (JOSE)
  - *JSON Web Signature* (JWS)
  - *JSON Web Encryption* (JWE)
  - *JSON Web Key* (JWK)

---

<sup>12</sup>RFC 7519 – <https://datatracker.ietf.org/doc/html/rfc7519>

# Estrutura do JWT I

Veja <https://jwt.io> e <https://www.base64decode.org>

- Um JWT é composto por três partes: **header**, **payload** e **signature**, separadas por pontos e codificadas em *base64url*
  - **Header** contém o tipo do token e o algoritmo de assinatura
  - **Payload** contém as declarações (*claims*)
  - **Signature** é a assinatura do token
- `header.payload.signature`

`eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJub21lIjoiaWF0IjE5MDE5MTIzLnR5cCI6IkpXVCJ9.eyJub21lIjoiaWF0IjE5MDE5MTIzLnR5cCI6IkpXVCJ9`

# Estrutura do JWT II

Veja <https://jwt.io> e <https://www.base64decode.org>

```
// Header
{
  "alg": "HS256", // HMAC com SHA-256
  "typ": "JWT" // Tipo do token
}
// Payload
{
  "nome": "Aluno",
  "matricula": 123456
}
// Signature - usando segredo "ifsc" no HMAC
HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), "ifsc")
```

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJub21lIjoiaWoiQWx1bm8iLCJtYXRoYWN1bGEiOiJyMzQ1Nn0.
EuVhoGzEDmyaL-2m5njiU23pLTHuKC21kfPOH_tcv8
```

# Curiosidades



# Entropia ruim permite recuperar senha perdida

<https://www.wired.com/story/roboform-password-3-million-dollar-crypto-wallet>

- Usuário do *RoboForm* (gerenciador de senhas) criou uma senha para sua carteira de criptomoedas, mas a perdeu
  - Solicitou ajuda de dois hackers e apenas soube informar que a senha foi gerada em 2013
  - De acordo com o log da carteira, foram movidos 43.6 BTC em 14/04/2013
- Engenharia reversa do *RoboForm* revelou que o gerador de números pseudoaleatórios usava a data e hora do computador do usuário como semente
  - Geraram todas senhas possíveis de 20 de abril a 1 de junho de 2013
- Descobriram que a senha foi gerada em 15/05/2013, às 16:10:40 UTC
  - <https://www.youtube.com/watch?v=o5lySpAkThg>

# Meta armazenava senhas de seus usuários em claro

<https://arstechnica.com/information-technology/2019/03/>

[facebook-developers-wrote-apps-that-stored-users-passwords-in-plaintext/](#)

- Desde 2012 a *Meta* armazenava senhas de seus usuários em texto claro
  - Banco com senhas foi consultado por 2.000 engenheiros da Meta por cerca de 9 milhões de vezes
- Em 2024<sup>13</sup> a *Meta* foi multada em US\$ 101 milhões pela Irlanda por violação do GDPR
- Até 2024 a Meta já recebeu mais de US\$ 2.3 bilhões de multas da União Europeia por violações de privacidade

---

<sup>13</sup><https://arstechnica.com/security/2024/09/>

[meta-slapped-with-101-million-fine-for-storing-passwords-in-plaintext](#)

# Aula baseada em



BISHOP, Matt. **Computer Security: Art and Science**. 1. ed.: Addison-Wesley, 2003. ISBN 0201440997.



GRASSI, Paul; GARCIA, Michael; FENTON, James. **NIST Special Publication 800-63-3 Digital Identity Guidelines**. 2020. <https://doi.org/10.6028/NIST.SP.800-63-3>.



MELLO, Emerson Ribeiro de *et al.* Autenticação e Autorização: antigas demandas, novos desafios e tecnologias emergentes. *In*: MINICURSOS do XXI Simpósio Brasileiro em Segurança da Informação e de Sistemas Computacionais (SBSeg). Porto Alegre, RS, set. 2022. DOI: <https://doi.org/10.5753/sbc.10710.3.1>.



SCHNEIER, Bruce. **Clique aqui para matar todo mundo**. 2020. Disponível em: <https://app.minhabiblioteca.com.br/reader/books/9788550808871>.